

Infografía

Módulo 7 — Godot 4 • Sesión 5

TileMapLayer: construir mundos con tiles

UPB • Gestión I/2026

Plan de la clase

1. **¿Qué es un tile?** — tileset, atlas, celda.
2. **El flujo a mano** — importar, crear el TileSet, pintar.
3. **Colisión por tiles** — el mundo que choca.
4. **Varias capas** — suelo + paredes.
5. **Mapa por código** — un nivel es una tabla de números.
6. **Autotile / terrenos** — los bordes se resuelven solos.
7. **Leer el mapa** — el tile sabe cosas (ejercicio).

Sesión 4: cómo se VE un personaje. Hoy: cómo se ve, y choca, todo el MUNDO por el que camina.

La pregunta del día

Un nivel tiene miles de pastos, paredes y caminos.
¿Vamos a poner **miles de Sprite2D** a mano?

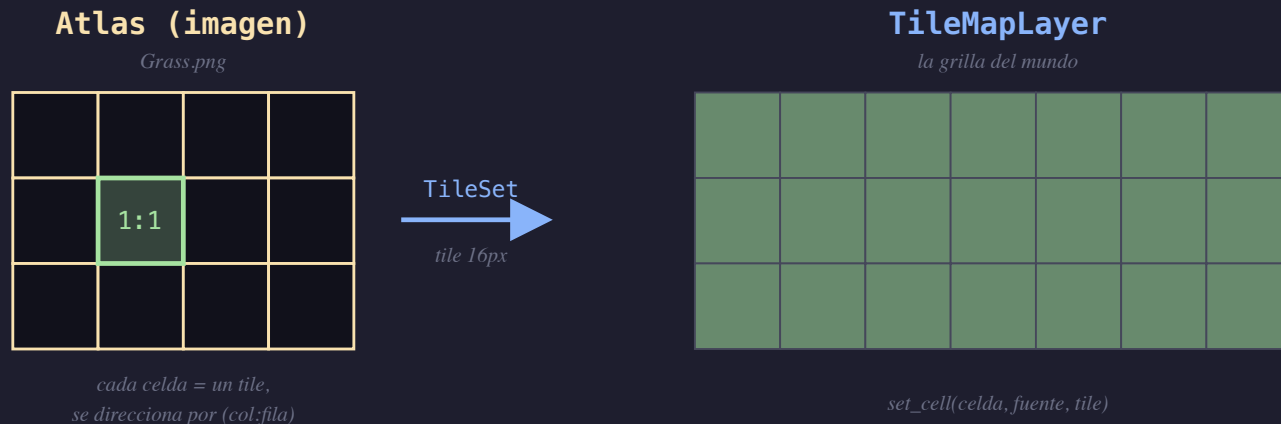
No. Una pieza, una grilla, y se repite. Eso es un tilemap.

Parte 1 — tileset, atlas, celda

Tres palabras nuevas, una sola idea:

- **Atlas** — una imagen cortada en una **grilla** de piezas (los tiles).
- **TileSet** — el recurso que dice "estas piezas miden 16px y vienen de esta imagen".
La **caja de piezas**.
- **TileMapLayer** — el **nodo** donde pintás esas piezas sobre una grilla. El **mundo**.

una imagen + una grilla = piezas que se repiten



Parte 2 — el flujo a mano (escena 01)

El flujo que vas a repetir toda tu vida en Godot:

1. Arrastra `Grass.png` al proyecto (Godot lo **importa**).
2. Seleccioná el `TileMapLayer` → en el inspector, `Tile Set` → **New TileSet**.
3. Poné **tile size = 16×16**.
4. Abajo, panel **TileSet** → `+` → arrastrá la textura como **fuentes (atlas)**. Godot crea las celdas.
5. Panel **TileMap** → elegí un tile → **pintá** en la pantalla.

No hay código todavía. Esto es 100% editor — el flujo del artista.

Parte 3 — colisión por tiles (escena 02)

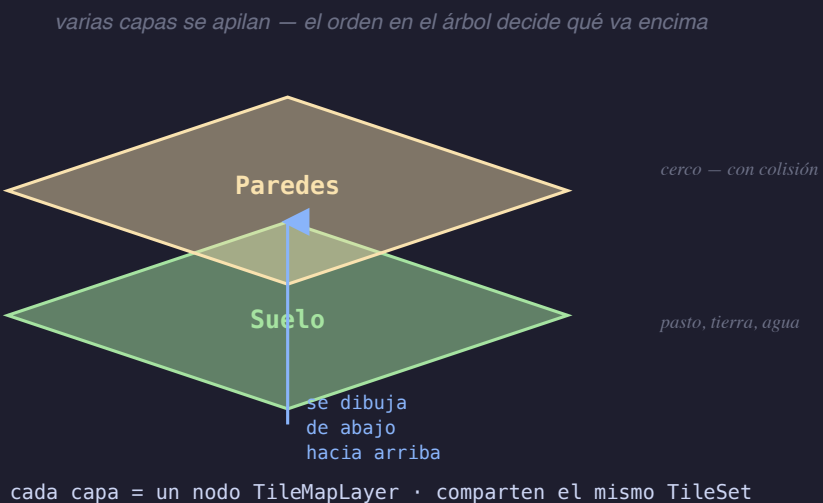
Las paredes tienen que **chocar**. La colisión no va en cada tile pintado: va una vez en el **TileSet**.

1. TileSet → **Physics Layers** → agregá una capa (capa mundo).
2. Seleccioná el tile de cerco → pestaña **Physics** → dibujá el polígono (un cuadrado de 16×16).
3. Pintá el cerco en la capa **Paredes**.

El jugador (de la sesión 2) ya está en la capa jugador con mask mundo : choca solo, sin tocar su código.

Parte 4 — varias capas

Un mundo real tiene **suelo** (no choca) y **paredes/objetos** (chocan, van encima). Dos cosas distintas → **dos TileMapLayer**.



Mismo TileSet, distinto nodo. El orden en el árbol = qué se dibuja encima.

Parte 5 — el mapa es DATO (escena 03)

Pintar a mano está bien para un nivel. ¿Y para 50? Un mapa es una **grilla**, y una grilla es un **array 2D**.

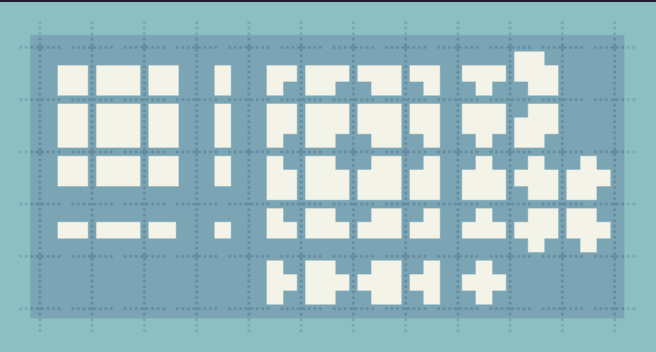
```
const TILES := { 0: Vector2i(1,5), 1: Vector2i(2,5), 2: Vector2i(0,0) }
const MAPA := [
    [2, 0, 0, 0, 0, 1],
    [0, 1, 1, 0, 0, 0],
]

func _ready() -> void:
    for y in MAPA.size():
        for x in MAPA[y].size():
            set_cell(Vector2i(x, y), 0, TILES[MAPA[y][x]])
```

set_cell(celda, id_de_fuente, tile_en_el_atlas) — el corazón de todo.

Parte 6 – autotile: los bordes solos (escena 04)

Pintar el borde de una zona (esquinas, lados, diagonales) a mano es horrible. Un **terrain set** lo hace por vos: mira los **vecinos** de cada celda y elige el tile que encaja.



```
set_cells_terrain_connect(  
    celdas, terrain_set, terrain)
```

Vos decís *"esta zona es pasto"*.
Godot resuelve, casillero por casillero, **qué borde va dónde** mirando el patrón de vecinos (el *bitmask*).

Parte 7 — el tile sabe cosas (ejercicio 05)

Hasta ahora **escribimos** el mapa. Ahora lo **leemos**: cada tile puede llevar datos propios (un **custom data layer**).

```
var celda := mapa.local_to_map(mapa.to_local(global_position))
var datos := mapa.get_cell_tile_data(celda) # puede ser null
if datos and datos.get_custom_data("tipo") == "agua":
    factor = 0.4 # el jugador se frena en el agua
```

🎓 Tu turno: completá los 3 TODO de leer_mapa.gd.

El premio: mundo jugable (escena 06)

Todo junto, armado por código:

- **Suelo:** pasto + camino + estanque.
- **Paredes:** un cerco que choca.
- El jugador camina y la **cámara lo sigue**.

Lo que aprendiste hoy:

- atlas → TileSet → TileMapLayer
- colisión por tiles
- capas
- `set_cell` (mapa = dato)
- autotile / terrenos
- leer el mapa (custom data)

Resumen

Un **tile** es una pieza.

Un **TileSet** es la caja de piezas (+ colisión, + datos).

Un **TileMapLayer** es el mundo donde las pintás —
a mano o por código.

El mundo no se dibuja: se construye con piezas que se repiten.